

유전 프로그래밍 거래규칙 생성기

코드 성능·구조 개선

— Python 자료구조·제너레이터·클래스 설계·데코레이터·벤치마크 적용 —

과목 : 파이썬 프로그래밍

대상 코드 : GenTradingRuleWithGP (DEAP 기반 GP 거래전략 생성)

학번 : 202550473

이름 : 정용한

제출일 : 2026-06-21

1. 연구 코드 소개

본 과제는 본인 연구에서 사용하는 **GenTradingRuleWithGP** 코드를 대상으로 한다. DEAP 유전 프로그래밍(GP)으로 기술적 지표 기반 매수/매도 규칙 트리를 진화시켜, 수익을 내는 거래 전략을 자동 탐색하는 파이프라인이다.

1.1 연구 맥락과 파이프라인 위치

- **입력**: 지표/조건 풀 크기, GP 하이퍼파라미터(개체수·세대·교차·변이율·트리 깊이), 백테스트 설정.
- **처리**: ① 지표·조건 풀 생성(ConditionManager) → ② 강타입 GP 트리 진화 (선택·교차·변이) → ③ 각 개체(트리)를 전략 JSON으로 **파싱** → ④ **검증/정리** → ⑤ 백테스트로 적합도 평가 → ⑥ 캐시.
- **출력**: 최적 전략 JSON, 진화 곡선/적합도 분포 시각화, 로그.
- **파이프라인 위치**: 전략 **탐색(search)** 단계. 평가 1회는 외부 백테스트 엔진 (BullTrader, bt4) 호출이며, 개체수×세대만큼 수만~수십만 번 반복된다.

1.2 개선 대상으로 선택한 이유

- 진화 루프가 **핫 루프**다. 개체마다 파싱·검증·해시·캐시가 반복되어, 평가 외 오케스트레이션 코드의 비용이 전체 실행 시간에 직접 누적된다.
- 실제 적합도 평가(bt4)는 외부 독점 엔진과 시장 데이터에 의존해 **재현이 불가능**하다. 과제 지침(synthetic 허용)에 따라, 평가 leaf 를 **결정적 합성 평가기**로 대체하여 GP 루프 전체를 재현 가능하게 만들고, 최적화 대상은 그 주위의 **파이썬 코드 구조**로 한정했다 (데이터·모델 축소가 아닌 코드 구조 개선).

2. 기존 코드의 문제점 분석

분류	위치	문제
중복 연산	strategy/parser.py	<code>str(individual)</code> (트리 직렬화, $O(N)$)를 buy/sell용으로 개체당 2회 호출
정규식 재검 파일	parser/formatter/helpers	변환·추출 정규식(~13개)을 호출마다 인라인 <code>re.sub/re.match</code>
코드 중복	parser.py	buy/sell 추출 함수가 슬라이스 한 줄만 다르고 동일(괄호 스캔 복붙)
자료구조	indicator_configs.py	<code>OHLCV_SOURCES</code> 가 list → membership test $O(n)$ 이 피연산자마다 반복
SRP 위반	main.py	<code>main()</code> 한 함수(~130줄)에 설정·상태·루프·통계·로깅·시각화·저장 혼재
반복 순회	main.py	세대 통계에서 <code>fits</code> 리스트를 4회 순회(2-pass 표준편차)
전역 상태	gp/evaluator.py	<code>_strategy_cache/_bt4_config/_bt4_adapter</code> 가 모듈 전역 → 테스트·실험 조건 변경 곤란
재현성	requirements.txt	<code>conda@ file://</code> 경로 다수 → 타 환경 <code>pip install</code> 불가

3. 적용한 수업 개념과 선택 근거

지침 5.2의 A~D 네 범주를 모두 적용했다(≥ 3 요건 초과). 각 선택의 근거를 함께 기술한다.

A. 자료구조 및 복잡도

- **frozenset**: OHLCV_SOURCES 멤버십을 $\text{list}(O(n)) \rightarrow \text{frozenset}(O(1))$. 지배 연산이 "피연산자가 OHLCV인가" 판정이라, 개체·조건마다 반복되는 선형탐색을 상수시간으로. `random.choice` 가 필요한 곳은 인덱싱 가능한 순서 리스트를 유지(난수열 보존).
- **defaultdict(list)**: 지표 그룹화의 `if k not in d: d[k]=[]` 관용구 제거.
- **단일 패스 통계**: 합·제곱합·최대·최소를 한 번에 누적(2-pass $\rightarrow$ 1-pass). 모집단 표준편차($\div n$) 보존.

B. 이터레이터 · 제너레이터 · Lazy

- **진화 루프를 제너레이터로**: `EvolutionRunner.run()`이 세대별 통계를 `yield` $\rightarrow$ 호출부가 lazy 소비(실시간 로깅/조기종료). 중간 리스트 불필요.
- **lazy genexpr**: 적합도를 리스트로 `materialize` 하지 않고 제너레이터로 단일 패스 통계에 투입.
- **중복 루프 통합**: 파서의 `buy/sell` 시스템 생성 루프를 하나의 헬퍼로 통합.

C. 클래스 설계 · 단일책임원칙

- **GPConfig** @dataclass(`frozen=True`, `slots=True`): 흩어진 전역 상수를 불변 설정 객체로. `frozen`=재현성 보장, `slots`=메모리·오타 방지, `replace`로 크기 스윙.
- **책임 분리**: `EvolutionRunner`(루프)·`StatsCollector`(통계)·`EvolutionState`(상태)·`StrategyEvaluator`(파싱 $\rightarrow$ 검증 $\rightarrow$ 캐시 $\rightarrow$ 평가)· 파일/시각화 유틸로 분해. `main()`은 ~40줄 조립으로 축소.
- **Protocol + Factory**: `BacktestEvaluator(Protocol)` 인터페이스 $\rightarrow$ `Synthetic/BT4` 두 구현을 `make_evaluator(name)` 로 교체. 평가 방식 변경이 호출부 한 줄로 가능(원래는 함수 본문 수정 필요).

D. 데코레이터

- **functools.lru_cache**: 순수 헬퍼 `extract_base_indicator_name` 등. 지표 이름이 반복 등장 $\rightarrow$ 적중률 높아 정규식/문자열 연산을 건너뛴.
- **cache_by_hash(stateful·parameterized)**: 전략 내용 해시 기반 캐시. `lru_cache`가 인자 identity로 캐싱해 매번 새 트리 객체엔 무의미한 반면, 서로 다른 트리도 동일 전략이면 적중.
- **validate_individual**(가드)·**retry**(불안정 외부 평가)·**timeit/log_call**(계측, 플래그로 no-op 토글). 모두 `functools.wraps`로 메타데이터 보존.

4. 개선 과정 (핵심 변경)

4.1 파서: 트리 직렬화 1회 + 중복 제거 + 정규식 사전 컴파일

```
# BEFORE - buy/sell 각각 str(individual) 호출(2회), 정규식 인라인
buy_op = _analyze_tree_structure(individual, "buy") # str(individual) (1)
sell_op = _analyze_tree_structure(individual, "sell") # str(individual) (2)
result = re.sub(rf"{system_type}_system(\d+)", r"system\1", s) # 매 호출 컴파일
```

```
# AFTER - 1회 직렬화 후 공유, 정규식은 모듈 로드 시 컴파일
_RE_SYSTEM = {"buy": re.compile(r"buy_system(\d+)"), "sell": re.compile(r"sell_system(\d+)")}
def _analyze_both(individual):
    tree_str = str(individual) # ← 단 1회
    buy_raw, sell_raw = _split_strategy_args(tree_str) # 괄호 스캔도 1회
    return _convert(buy_raw, "buy"), _convert(sell_raw, "sell")
```

4.2 거대한 main() → 제너레이터 기반 Runner (SRP)

```
# BEFORE - main() 안에 루프·통계·로깅·시각화·저장 혼재
for g in range(N_GENERATION):
    offspring = ...; # 선택/교차/변이
    fits = [ind.fitness.values[0] for ind in pop] # 리스트
    max_fit=max(fits); avg=sum(fits)/len(pop); std=(sum((x-avg)**2 ...)) # 4회 순회
    logging.info(...); stats_history.append({...})
```

```
# AFTER - 루프는 통계를 yield, 호출부는 lazy 소비
class EvolutionRunner:
    def run(self) -> Iterator[GenerationStats]:
        for g in range(cfg.n_generation):
            ... # 선택/교차/변이/평가
            yield self.stats.summarize(self.state.fitness_values(), g+1, dur) # lazy genex
```

4.3 캐시: 전역 dict → 인스턴스 + 내용 해시 데코레이터

```
def cache_by_hash(hash_fn): # parameterized + stateful 데코레이터
    def deco(func):
        @functools.wraps(func)
        def wrapper(self, payload, *a, **k):
            key = hash_fn(payload)
            if key in self._cache: self._hits += 1; return self._cache[key]
            self._misses += 1; r = func(self, payload, *a, **k); self._cache[key]=r; return r
        return wrapper
    return deco
```

4.4 고려했으나 적용하지 않은 대안 · trade-off

- **Welford 1-pass 표준편차**: 수치 안정성은 우수하나 원소당 연산↑. 현재 값 범위에선 sum/sumsq 단일 패스로 충분 → 후자 채택(대안 명시).
- **multiprocessing Pool**: 합성 평가기는 가벼워 IPC/pickling 오버헤드가 지배적이고 워커별 RNG·캐시 분리로 **비결정적**이 된다. 공정·재현 벤치마크를 위해 순차를 기본으로 하되, pickling-aware PoolMap(Pool initializer)은 시연용으로 구현.
- **deepcopy 검증**: 메모리·시간 증가만 유발. 원본의 `shallow copy()` 의미 보존.

5. 최적화 결과

1.23×

parse (평균)

1.13×

validate (평균)

10.4×

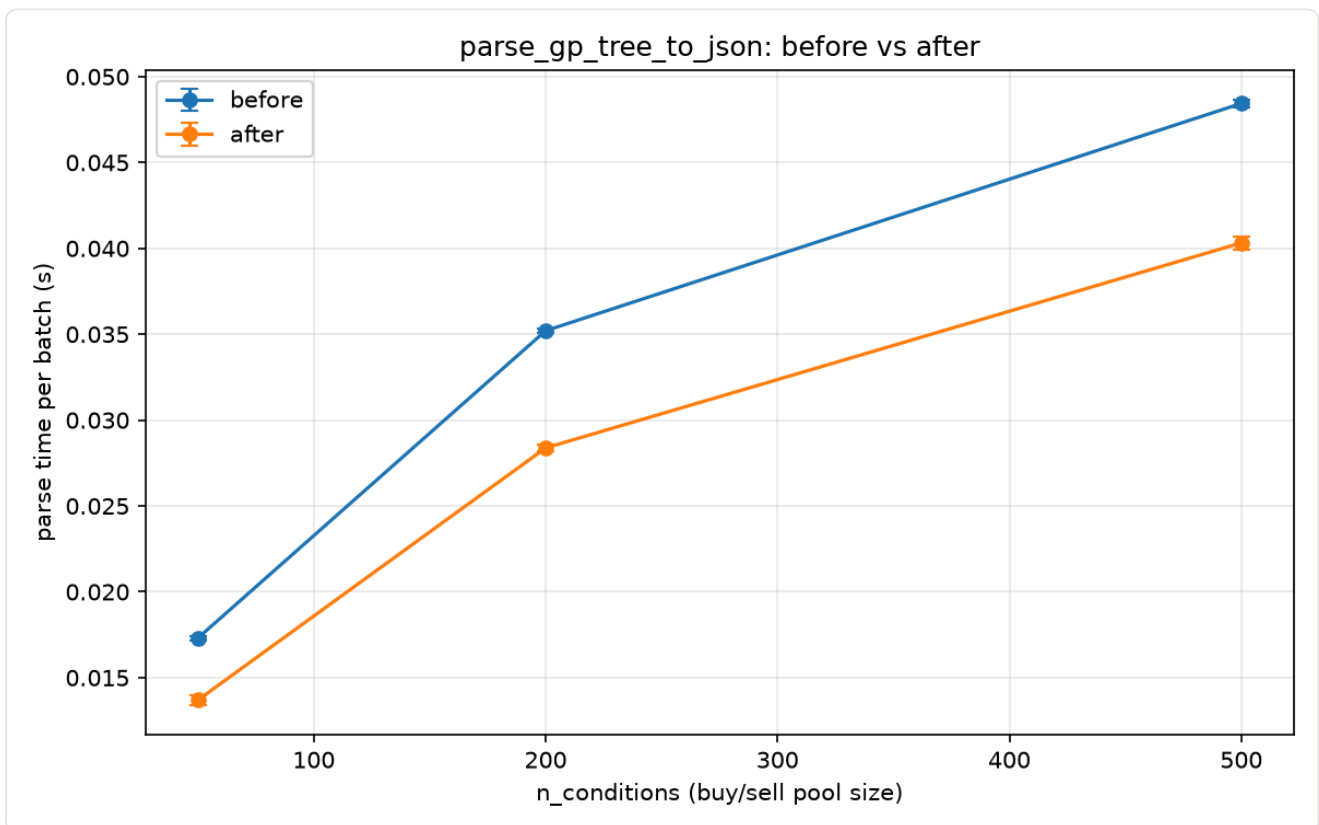
extract (lru_cache)

1.18×

전체 진화 (평균)

5.1 마이크로벤치: parse_gp_tree_to_json (입력 크기별)

n_conditions	before (ms)	after (ms)	speedup	peak KB (b/a)	출력 동일
50	17.303 ± 0.142	13.692 ± 0.267	1.26×	78 / 69	True
200	35.205 ± 0.100	28.379 ± 0.176	1.24×	249 / 239	True
500	48.442 ± 0.206	40.319 ± 0.371	1.20×	390 / 372	True



5.2 마이크로벤치: validate_and_clean_strategy

n_conditions	before (ms)	after (ms)	speedup	peak KB (b/a)	출력 동일
50	0.935 ± 0.033	0.806 ± 0.016	1.16×	3 / 3	True

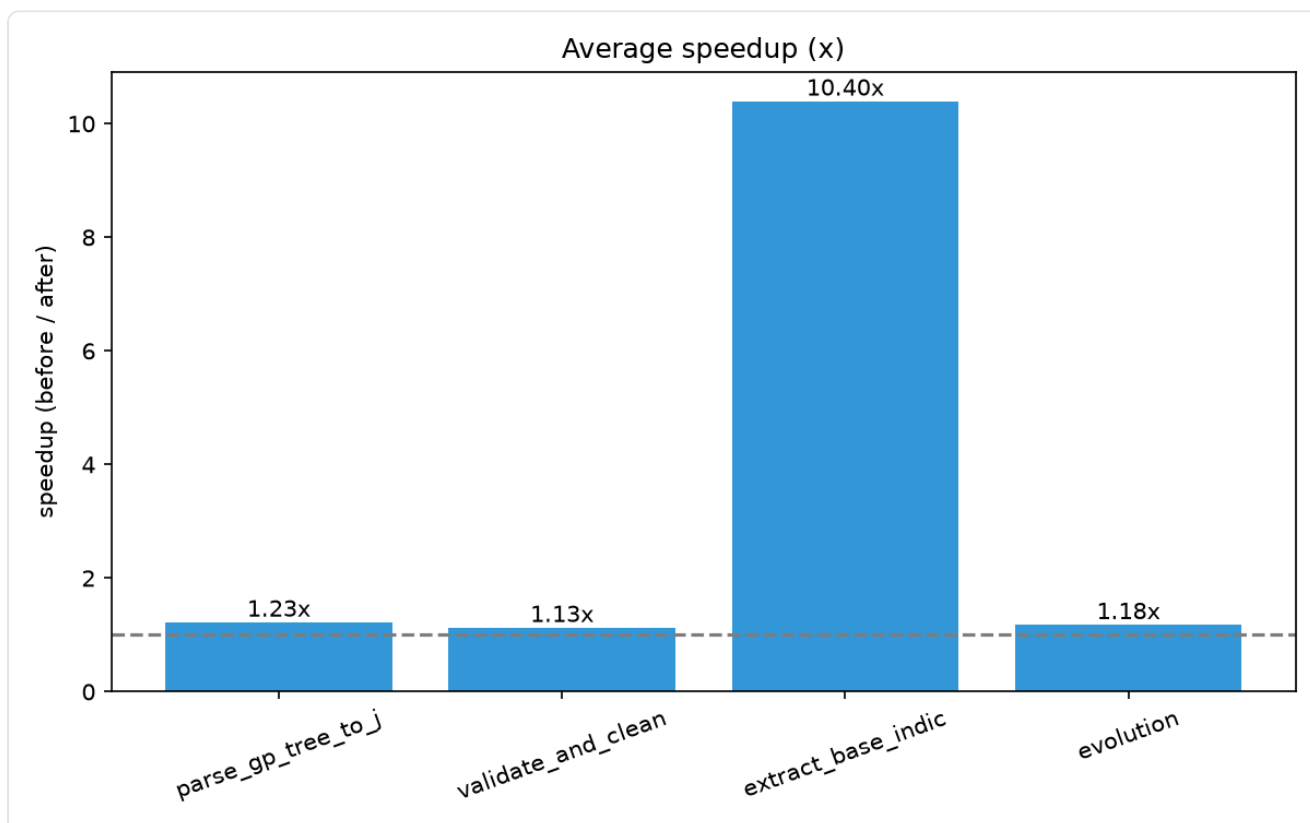
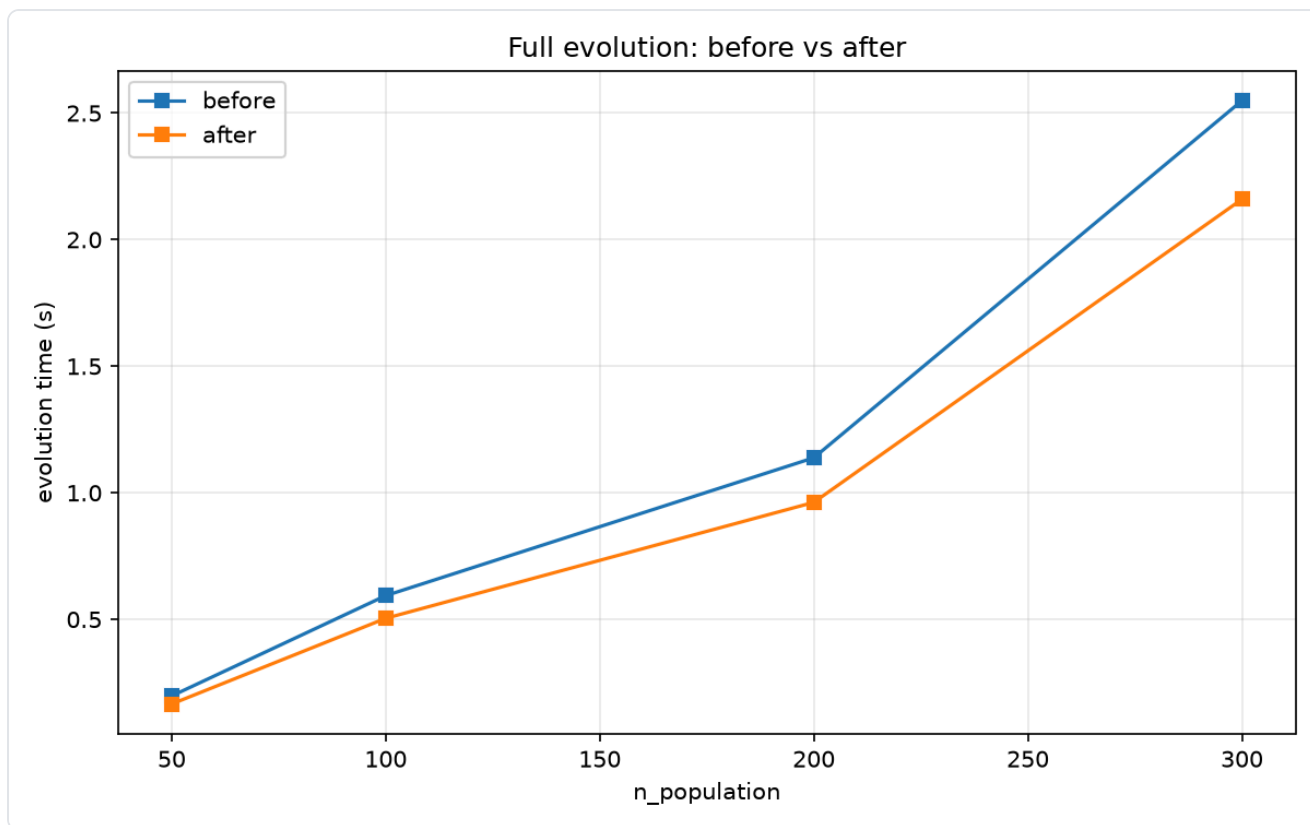
200	1.726 ± 0.039	1.565 ± 0.042	1.10×	11 / 11	True
500	1.519 ± 0.091	1.327 ± 0.054	1.14×	8 / 8	True

5.3 마이크로벤치: extract_base_indicator_name (5만 회 호출)

지표 이름이 반복 등장하는 분포에서 **lru_cache** 효과. before 21.43 ms → after 2.06 ms (**10.4×** 향상).

5.4 매크로벤치: 전체 진화 (합성 평가기, 순차)

개체수 × 세대	before (ms)	after (ms)	speedup	peak KB (b/a)	고유 전략	궤적 동일
50 × 10	197.4 ± 0.9	165.7 ± 1.1	1.19×	549 / 560	360	True
100 × 20	593.8 ± 1.0	504.2 ± 5.5	1.18×	697 / 704	1524	True
200 × 20	1138.3 ± 3.9	962.3 ± 3.3	1.18×	1078 / 1068	2928	True
300 × 30	2549.0 ± 6.9	2159.5 ± 9.6	1.18×	1949 / 1887	6920	True



5.5 구조·재사용성 개선

항목	before	after
----	--------	-------

main() 라인수	~130줄(다중 책임)	~40줄(조립)
평가 방식 교체	함수 본문 수정	호출부 1줄(make_evaluator)
설정 변경	전역 상수 직접 수정	GPConfig 필드/replace
캐시 상태	모듈 전역	인스턴스(테스트·병렬 안전)
파서 추출 함수	2개 중복	1개 통합
정규식 컴파일	호출마다	모듈 로드 1회

6. 결과 해석 및 한계

6.1 왜 이런 결과가 나왔는가

- **extract_* 10.4x**: 가장 큰 향상. 동일 인자 재호출이 매우 잦아 lru_cache 적중률이 높고, 정규식 매칭 자체를 건너뛰었다. 입력 다양성이 낮을수록 효과가 크다(캐시의 본질).
- **parse 1.23x**: `str(individual)` 2→1회가 핵심 기여. 트리가 클수록 직렬화 비중이 커져 효과가 두드러진다. 단, 파서 비용의 상당 부분은 buy/sell 풀 전체를 순회하는 `_build_systems(O(n_conditions))`라, str 절감만으로 2배가 되지 않는다 → 1.2배대가 타당.
- **전체 진화 1.18x**: 입력 크기와 무관하게 일정한 비율의 향상. 파싱·검증·헬퍼가 세대마다 누적되므로, 마이크로 향상이 전체로 안정적으로 전이된 결과.
- **메모리**: 큰 규모(p300×g30)에서 peak이 소폭 감소(1949→1887KB). 중간 리스트 제거 (lazy genexpr)·중복 객체 감소의 효과지만, 지배 메모리는 개체군 자체라 절감폭은 제한적.

6.2 한계와 trade-off

- **복잡도↑**: 클래스·Protocol·데코레이터 도입으로 파일 수가 늘었다. 소규모 1회 실행에는 과한 구조일 수 있으나, 반복 실험·확장·테스트에서 회수된다.
- **데코레이터 오버헤드**: 향상 커진 `@timeit/@log_call`은 핫패스에 프레임을 더해 벤치마크를 오염시킬 수 있어, 플래그로 no-op 토글하도록 설계했다.
- **합성 평가기**: 실제 bt4 백테스트가 아니므로 절대 성능이 아닌 코드 구조 비교에 초점. 실 엔진에선 평가 leaf가 지배적이라 오케스트레이션 향상 비중은 작아질 수 있다(동일 인터페이스로 교체 가능하게 설계).
- **추가 개선 여지**: 파서를 문자열 재파싱이 아닌 트리 순회 기반으로 재작성하면 정규식 파이프라인 자체를 제거할 수 있다(가장 큰 잠재 이득). 캐시 직렬화 비용(`json.dumps`)도 구조적 키로 대체 가능.

부록. 측정 환경 및 재현

항목	값
OS	macOS-26.5-arm64-arm-64bit
Python	3.11.14
CPU	arm (12 cores)
주요 라이브러리	deap 1.4.3, numpy 2.3.3, pandas 2.3.2
multiprocessing start	spawn (벤치마크는 순차)
random seed	42 (고정)
반복/워밍업	repeats=7, warmup=2
git commit	4da03fb
측정 시각	2026-06-13 21:56:07

재현: `uv venv --python 3.11 && uv pip install -r requirements.txt` 후 `python benchmark/run_benchmark.py`. 모든 행에서 before/after 출력·궤적 동일성(회귀 게이트)을 assert 한다. 전/후 코드는 `src/before`, `src/after` 로 분리.